THE C LANGUAGE · PRACTICE SERIES

Pointers in C — the solutions

&

All ten exercises solved and commented, with the why behind each line. Read after a real attempt.

```
while ((*d++ = *src++) != '\0') ;
```

10 commented solutions

compiles clean at -Wall -Wextra

+ the five classic pointer bugs

SOLUTION 01 · BEGINNER

Your first pointer

```c
int main(void)
{
    int x = 42;
    int *p = &x;  /* p = the address of x */

    printf("x  = %d\n", x);
    printf("&x = %p\n", (void *)&x);
    printf("p  = %p\n", (void *)p);
    printf("*p = %d\n", *p);

    *p = 100;  /* write through the pointer */
    printf("x  = %d\n", x);
    return 0;
}
```

WHY IT WORKS

- **p** and **&x** print the same value: p literally contains the location of x.
- ***p = 100** writes into x's bytes — x changes without ever being named.
- **%p** expects a void * — the cast is the portable idiom.

OUTPUT

```
x  = 42  ·  *p = 42
x  = 100
```

includes: stdio.h · compiles clean at -Wall -Wextra

SOLUTION 02 · BEGINNER

Pointers that point nowhere

```c
void safe_print(const int *p)
{
    if (p != NULL)  /* guard before the star */
        printf("value: %d\n", *p);
    else
        printf("(null pointer)\n");
}

int main(void)
{
    int x = 7;
    safe_print(&x);
    safe_print(NULL);
    return 0;
}
```

WHY IT WORKS

- The guard runs **before** any *p — the only order that helps.
- NULL is detectable; stack garbage is not. An uninitialized pointer passes the test and still crashes — hence "initialize at birth".
- **const int ***: callers can trust you not to write — and the compiler enforces it.

OUTPUT

```
value: 7
(null pointer)
```

includes: stdio.h

SOLUTION 03 · BEGINNER

Functions that write back

```c
void swap(int *a, int *b)
{
    int tmp = *a;
    *a = *b;
    *b = tmp;
}

int main(void)
{
    int u = 3, v = 7;
    printf("before  u=%d v=%d\n", u, v);
    swap(&u, &v);  /* hand over the addresses */
    printf("after   u=%d v=%d\n", u, v);
    return 0;
}
```

WHY IT WORKS

- Inside swap, ***a** and ***b** are u and v themselves — three assignments through their addresses.
- The by-value version swaps two **copies**; they die when the function returns.
- Same pattern as scanf("%d", &n): out parameters.

OUTPUT

```
before  u=3 v=7
after   u=7 v=3
```

includes: stdio.h

SOLUTION 04 · INTERMEDIATE

Walking an array

```c
int sum(const int *a, size_t n)
{
    int total = 0;
    for (const int *p = a; p < a + n; p++)
        total += *p;
    return total;
}

int main(void)
{
    int t[] = {2, 4, 6, 8, 10};
    size_t n = sizeof t / sizeof *t;
    printf("sum = %d\n", sum(t, n));
    return 0;
}
```

WHY IT WORKS

- At the call, **t** decays to **&t[0]** — inside, the array is just an address.
- **p++** really adds sizeof(int) bytes; the type system scales it for you.
- **a + n** may be formed and compared — never dereferenced.
- Index flavor, same thing: total += *(a + i);

OUTPUT

```
sum = 30
```

includes: stdio.h

SOLUTION 05 · INTERMEDIATE

Strings, one char at a time

```
size_t my_strlen(const char *s)
{
    const char *p = s;
    while (*p)  /* until '\0' */
        p++;
    return (size_t)(p - s);
}

char *my_strcpy(char *dst, const char *src)
{
    char *d = dst;
    while ((*d++ = *src++) != '\0')
        ;
    return dst;
}

/* in main: */
char buf[32];
my_strcpy(buf, "pointers are easy");
printf("%s (%zu)\n", buf, my_strlen(buf));
```

WHY IT WORKS

- **p - s** counts the elements stepped over: pointer difference.
- ***d++ = *src++** copies one char and advances both sides; the copied char is also the loop test, so '\0' is copied — then the loop stops.
- Returning **dst** allows chaining, mirroring the real strcpy.

OUTPUT

```
pointers are easy (17)
```

includes: stdio.h

SOLUTION 06 · INTERMEDIATE

A pointer to a pointer

```c
void point_to_max(int **sel, int *a, int *b)
{
    *sel = (*a ⩾ *b) ? a : b;
}

int main(void)
{
    int x = 3, y = 9;
    int *m = NULL;

    point_to_max(&m, &x, &y);
    *m = 0;  /* y becomes 0 */
    printf("x=%d y=%d\n", x, y);
    return 0;
}
```

WHY IT WORKS

- One line does it all: ***sel = …** stores an **address** into the caller's m. No int is copied.
- Three levels: **sel** is where m lives · ***sel** is m · ****sel** is the int.
- After the call m equals **&y**, so *m = 0 zeroes y — x never moves.
- Same law as swap: to mutate a thing, receive a pointer to that thing — even when the thing is itself a pointer.

OUTPUT

```
x=3 y=0
```

includes: stdio.h

SOLUTION 07 · INTERMEDIATE

The heap: malloc and free

```c
int main(void)
{
    size_t n = 8;
    int *a = malloc(n * sizeof *a);
    if (a == NULL) {
        fprintf(stderr, "out of memory\n");
        return 1;
    }
    for (size_t i = 0; i < n; i++) {
        a[i] = (int)(i * i);
        printf("%d ", a[i]);
    }
    printf("\n");
    free(a);
    a = NULL;  /* no dangling pointer */
    return 0;
}
```

WHY IT WORKS

- **sizeof *a** reads "the size of what a points to" — change the type once, the size follows.
- malloc is checked **before** first use; NULL means no memory.
- **a = NULL** after free: a late *a now crashes loudly instead of corrupting silently.
- valgrind's target sentence: "All heap blocks were freed — no leaks are possible."

OUTPUT

```
0 1 4 9 16 25 36 49
```

includes: stdio.h, stdlib.h

SOLUTION 08 · ADVANCED

A dynamic matrix

```c
int **make_matrix(size_t rows, size_t cols)
{
    int **m = malloc(rows * sizeof *m);
    if (!m) return NULL;
    for (size_t r = 0; r < rows; r++) {
        m[r] = malloc(cols * sizeof **m);
        if (!m[r]) {  /* clean up, no leak */
            while (r > 0) free(m[--r]);
            free(m);
            return NULL;
        }
        for (size_t c = 0; c < cols; c++)
            m[r][c] = (int)(r * cols + c);
    }
    return m;
}
```

includes: stdio.h, stdlib.h · main: m = make_matrix(3, 4); print; free_matrix(m, 3);

```c
void free_matrix(int **m,
                 size_t rows)
{
    if (!m) return;
    for (size_t r = 0; r < rows; r++)
        free(m[r]);  /* rows first */
    free(m);  /* the array last */
}
```

WHY IT WORKS

- Rows were malloc'd after m → freed before m.
- The unwind frees rows 0…r−1 exactly: the failure path leaks nothing.
- **m[r][c]** is *(*(m + r) + c) — two hops. Prints 0…11 as a 3 × 4 grid.

SOLUTION 09 · ADVANCED

Pointers to functions

```c
int add(int a, int b) { return a + b; }
int sub(int a, int b) { return a - b; }
int mul(int a, int b) { return a * b; }

int main(void)
{
    int (*ops[])(int, int) = { add, sub, mul };
    const char *names[] = { "add", "sub", "mul" };
    size_t n = sizeof ops / sizeof *ops;

    for (size_t i = 0; i < n; i++)
        printf("%s(6, 7) = %d\n",
            names[i], ops[i](6, 7));
    return 0;
}
```

OUTPUT

```
add(6, 7) = 13  ·  sub(6, 7) = -1  ·  mul(6, 7) = 42
```

WHY IT WORKS

- **{ add, sub, mul }** stores three addresses of code, not three calls.
- **ops[i](6, 7)** jumps wherever slot i points — dispatch without if/else.
- qsort's comparator is exactly this mechanism.

BONUS — QSORT COMPARATOR (STDLIB.H)

```c
int cmp_int(const void *a,
            const void *b)
{
    int x = *(const int *)a;
    int y = *(const int *)b;
    return (x > y) - (x < y);
}
qsort(t, n, sizeof *t, cmp_int);
```

SOLUTION 10 · ADVANCED

Capstone: a linked list

```c
struct node {
    int value;
    struct node *next;
};

void push_front(struct node **head, int v)
{
    struct node *n = malloc(sizeof *n);
    if (!n) { perror("malloc"); exit(1); }
    n->value = v;
    n->next  = *head;  /* link first */
    *head    = n;      /* then move head */
}

void print_list(const struct node *p)
{
    for (; p; p = p->next)
        printf("%d -> ", p->value);
    printf("NULL\n");
}
```

```c
void free_list(struct node **head)
{
    struct node *p = *head;
    while (p) {
        struct node *nx = p->next;
        free(p);
        p = nx;
    }
    *head = NULL;
}
```

WHY IT WORKS

- **&head** in main lets push_front rewrite the real head — exercise 6 at work.
- Link first, then move the head. The reverse order loses the list.
- One malloc per node, one free per node. Output: 3 -> 2 -> 1 -> NULL

RECAP · READ ME TWICE

The five bugs to watch for

1 **The wild read — declared, never initialized, dereferenced**
Fix: give every pointer a value at birth, a real address or NULL · exercises 1–2

2 **The NULL dereference — *p while p is NULL**
Fix: guard before the star; decide each function's NULL policy · exercise 2

3 **The leak — malloc without free**
Fix: pair them as you write; valgrind must say "no leaks are possible" · exercises 7–10

4 **Use-after-free and double free — the pointer outlived the memory**
Fix: free once, set the pointer to NULL right after · exercise 7

5 **The escaped local — returning the address of a local variable**
Fix: the frame dies at return — hand back heap memory, or fill a caller-supplied buffer

gcc -Wall -Wextra and **valgrind** catch most of these — make them part of "done".